

QUARTERLY AWARDS · INNOVATION

Enterprise Risk Management (ERM)

AI-assisted migration from Slate to Airbus DevX (React + OSDK)

[Team name] · [Presenter names] · [Date]

What ERM does and how it works

ERM helps stakeholders understand risks, decision bases and required actions. The ERM dashboard builds ERM reports from Active Risk Manager (ARM) data in Palantir Foundry Skywise.

Single place for ARM data

Live view of all ARM risks and opportunities, with role-based access

Custom dashboards

Group the risks and opportunities you need into dedicated dashboards

Live editing and trend following

Risk managers update key parameters and follow status over time

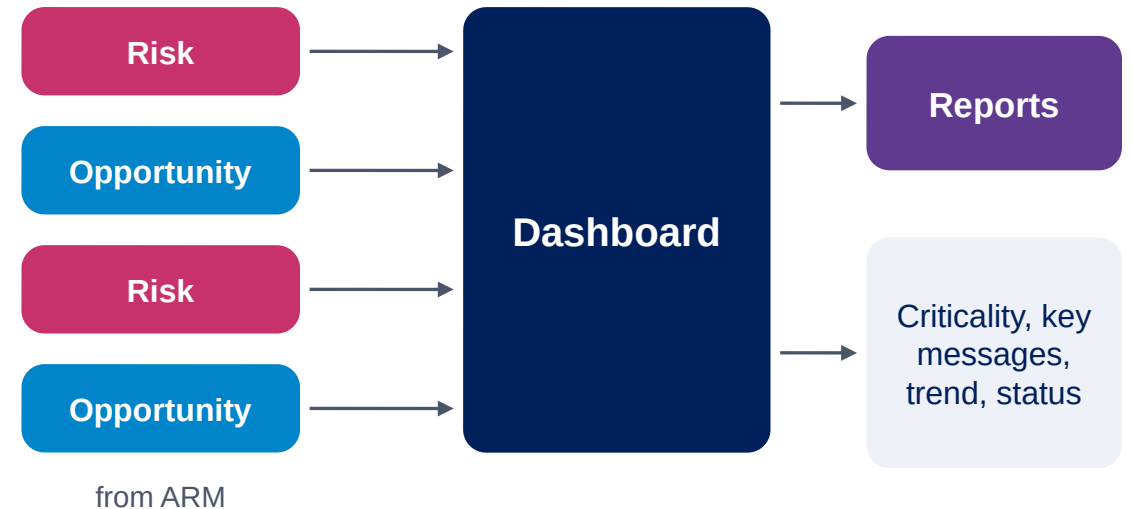
Structured release workflow

Formal, governed quarterly review and release

Cross-team collaboration

Share and co-manage individual risks across divisions

HOW DATA BECOMES A REPORT



DATA PATH



ERM adoption and user metrics

A migration that broke a feature would disrupt risk reporting for hundreds of risk managers in 6 countries

LIFETIME UNIQUE USERS

892

Across engineering, programmes and operations

MONTHLY ACTIVE PEAK

320+

Active risk managers per reporting month

DASHBOARDS CREATED

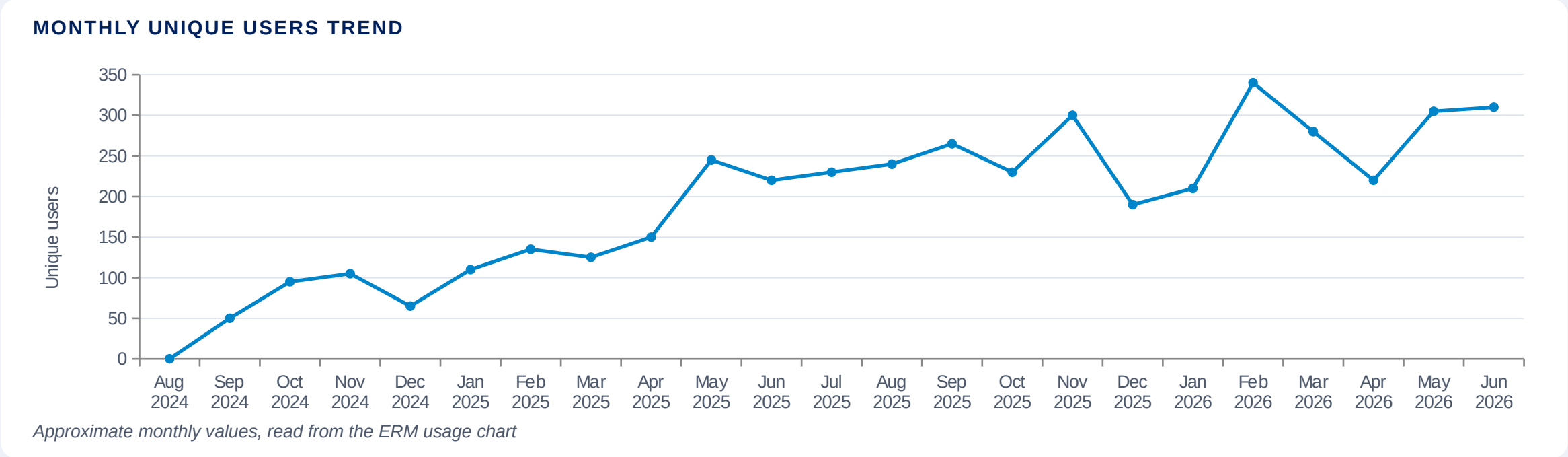
1,600+

Custom dashboards created to date

UPDATED DAILY

10+

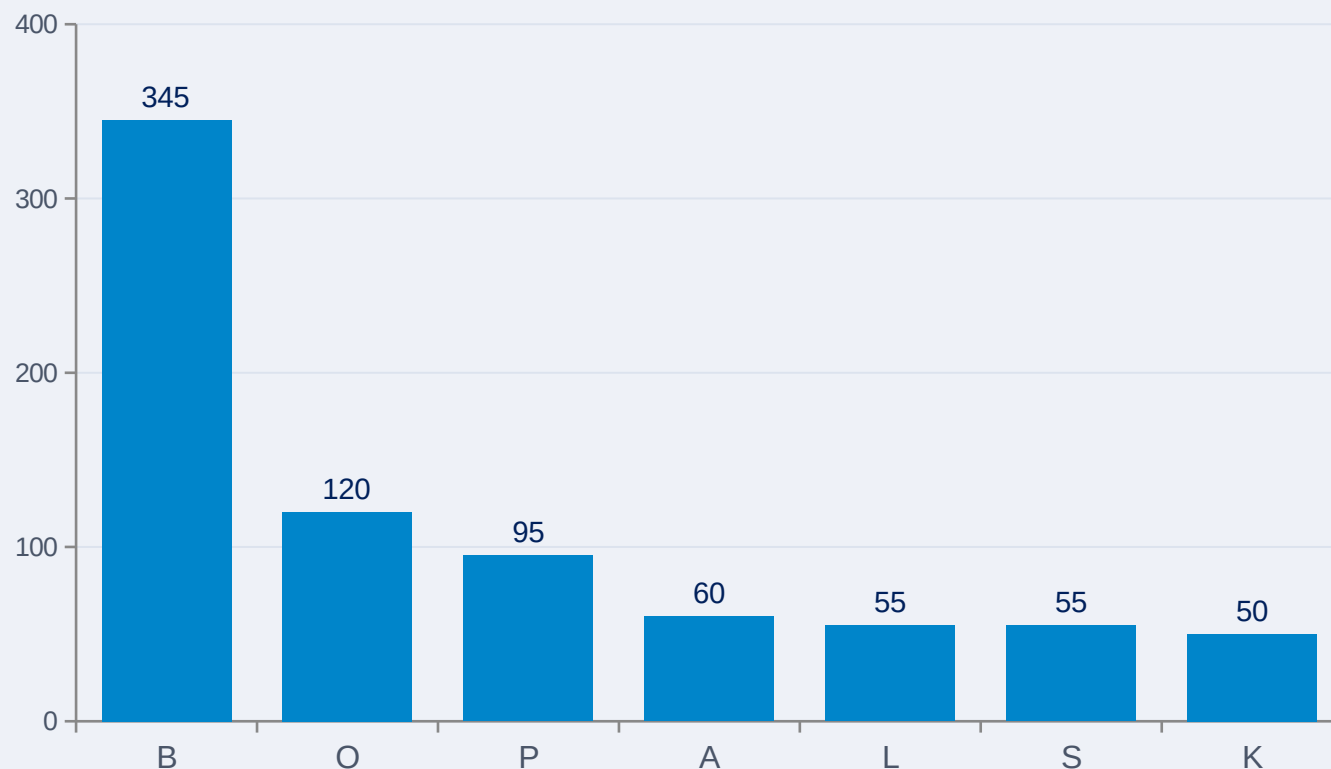
Maintained continuously by risk owners



Used across Airbus organisations and sites

More than half of ERM users work outside its largest organisation, across 6 countries

UNIQUE USERS BY ORGANISATION (SIGLUM 1)



Plus about 12 smaller organisations (D, M, Z, Q, G, F, H, I, V, E, T...). Approximate values, read from the ERM usage chart

LOCATION PROFILE

France

Blagnac Wings Campus 271, Saint-Martin 43 users

Germany

Hamburg Finkenwerder 205 users

United Kingdom

Broughton 32, Filton 26 users

Spain

Getafe 25 users

Canada

Mirabel 12 users

India

Bangalore 7 users

Why migrate: Slate had become the bottleneck

Tangled dependencies

One user action cascades through widgets, events, actions and variables

High regression risk

Changing one feature can break unrelated dashboard behaviour

Slow navigation

Variable trees are recalculated on every tab switch

No documentation

8 Slate exports, about 25 MB of configuration, no specs

One big undocumented app

All behaviour hidden in Slate configuration

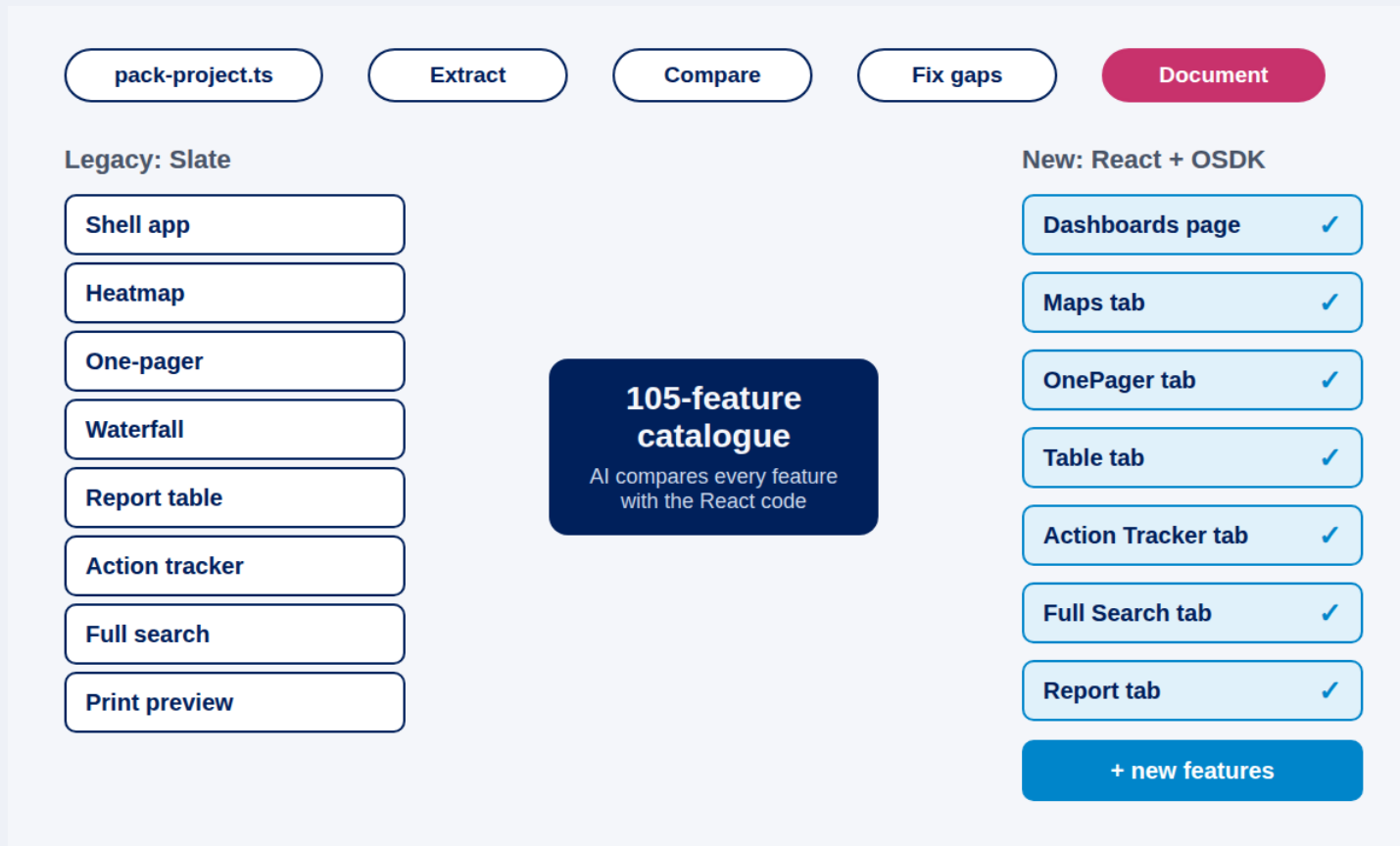
ERM Slate app

8 JSON exports · about 25 MB · no documentation

The Airbus DevX solution: React + OSDK

Clean, modular TypeScript components: isolated modules, safe updates, instant navigation

Our method: AI drafts, people check



pack-project.ts

Our script packs the whole React app, 75 files, into one dated file the AI reads in one go

1. Pack the context
2. Extract legacy features
3. Compare with the React code
4. Fix gaps, re-pack, re-check
5. Document the new app

The process in numbers

1 PI

Whole migration built in one SAFe Program Increment

105

legacy features recovered, with no docs

8

Slate exports analysed, about 25 MB

75 → 1

React files packed into one AI context

31

gaps found and prioritised by AI review

7

security and data issues caught in review

11

new features beyond the old app

0 → 10

documents: 8 diagrams and 2 guides

Effort saved: about 76%, or ~\$80K

~17

Manual estimate

sprints, $\approx$ 340 person-days $\approx$ \$105K

4

Actual with AI

sprints, $\approx$ 80 person-days $\approx$ \$25K

~\$80K

Saved

about 76% less effort: $(17 - 4) \div 17$

Where the AI saved the most time

Discovery

105 features catalogued from the exports, with no docs to read

Gap analysis

Every feature checked against the packed React code

Documentation

8 architecture and lifecycle diagrams produced alongside the work

Generic estimate, not a measured baseline: 105 features $\times$ (0.5 day discovery + 2 days build + 0.5 day parity testing) + 25 days documentation $\approx$ 340 person-days. 2 developers, 2-week sprints = 20 person-days per sprint; 1 PI = 4 sprints; \$38.50 per hour, 8 hours a day.

Faster to build, cheaper to run

11

new features Slate never had, built in the same PI

11 files

one change added app-wide notifications and auto-refresh

−97%

25 MB of Slate configuration became 0.8 MB of typed code

~\$10K

less maintenance effort every year (estimate)

ADDING A FEATURE IN SLATE

- Wire widgets, events, variables and functions across 8 exports
- Any change can break unrelated dashboards
- No documentation to know what depends on what

ADDING A FEATURE IN REACT + OSDK

- One isolated, typed component plus one OSDK call
- `pack-project.ts` gives AI the whole app as context
- Architecture and lifecycle diagrams show the impact

Estimates at \$38.50 per developer hour, 8 hours a day. Maintenance: rule of thumb of ~20% of build effort a year (340 person-days → 68 a year, ≈ \$21K), assumed halved by modular, typed, documented code (≈ \$10K a year saved).

What the business says

Feedback from ERM business stakeholders, September 2026

“

Thanks to a mix of regular teamwork with the business and strong IT skills, they have turned an idea into a user-friendly industrial [solution]... User interface and performance have significantly improved.

ERM Centre of Competence
Enterprise Risk Management

“

The Skywise ERM dashboard application is a tremendous addition to our ERM toolset... More and more users regularly prepare and facilitate risks & opportunities reviews with the application.

ERM stakeholder
[Role]

“

They built great new features that make navigation easier and give us a much clearer view of our R&O data. Whenever bugs or issues came up, they fixed them quickly and efficiently.

Risk and Opportunity Manager
Airbus Operations GmbH, Hamburg

Our contribution to the 2026 objectives

OUR PROJECT

EVIDENCE

2026 OBJECTIVE

AI-assisted migration of ERM

Slate to React + OSDK
2 developers · 1 PI
pack-project.ts

AI read 8 Slate exports and the packed React code: 105 features recovered, 31 gaps found

Delivered in 1 PI by 2 developers, about 76% less effort than a manual estimate

All 105 features tracked for parity; ERM rebuilt on a standard, maintainable stack, ready to leave Slate

pack-project.ts and the playbook let any team repeat the method

Leverage AI to solve business problems

India TFO · Digital: democratise AI

Deliver on time, on cost, on quality

Digital: optimise our digital resources

Ensure our operational performance

Digital: availability of systems for operations

Grow digital acumen and competencies

Digital: develop our people · India TFO

A playbook any team can reuse

Built together

- Business users confirmed the feature catalogue
- Data and security owners kept restricted views intact
- Backend and frontend fixes delivered together

Ready to transfer

- Proposed at the architecture sync, aligned with [Architect], for functions beyond DMS
- Other Slate apps in [business areas]
- Any low-code tool with little documentation
- Reusable as is: pack-project.ts, comparison template, diagrams

Pack. Extract. Compare. Fix. Document. Repeat for the next app.